eedi-mining-misconceptions-in-mathematics

6位 Solution | Nikita Babych

Rank	6
Team	Nikita Babich
Author	Nikita Babych
Topic ID	551565
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551565
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

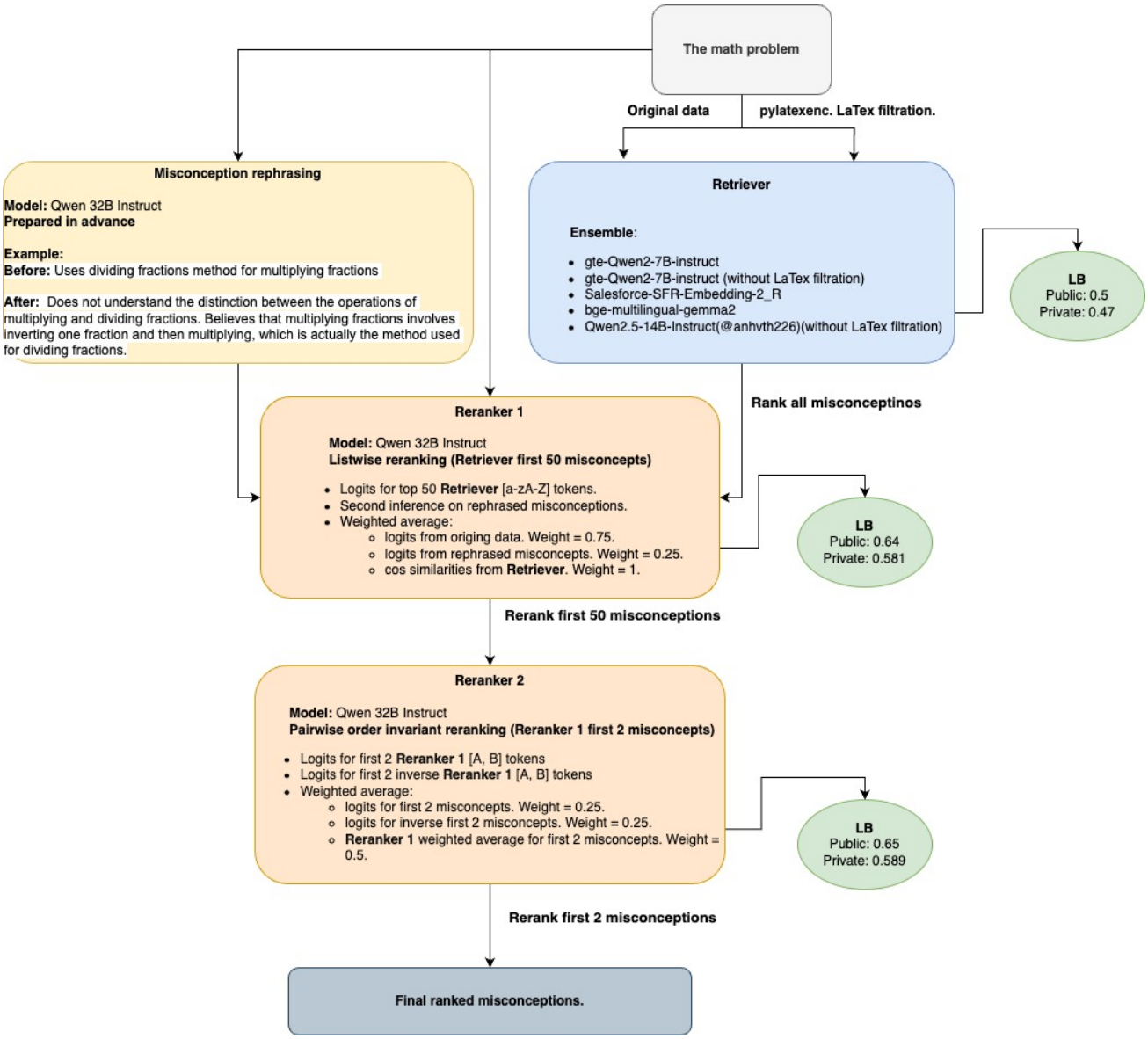
1年前に比較的本格的にKaggleコンペに参加し始めたときに自分で設定した目標、つまりsolo goldをついに達成できて、とても嬉しく思います。 以前にもpublicでsolo goldに届いたことは何度かありましたが、privateで生き残ることはできませんでした。今回は、ここ数か月の献身、汎化するsolutionに注力したこと、そしてpublicとprivateのデータ分布が比較的似ていたことによって、この目標を達成することができました。

感謝

このように興味深いコンペを開催してくださったKaggleとhostの皆様に感謝します。このコンペでは、distractorとの関連度によって数学のmisconceptionを効果的にrankingするために、さまざまなアプローチの推論能力を探る必要がありました。 特に、training dataで既出（seen）のmisconceptionと未出（unseen）のmisconceptionの数がバランスするようにpublic/private splitを設計してくださったことに感謝します（生き残ったgold圏の皆さんも同意してくれると確信しています）。これにより、開発したアプローチの頑健性がleaderboardに反映され、privateでのshuffleはごく穏やかなものにとどまりました。最近のいくつかのコンペを経て、この点には本当に感謝すべきだと感じています :)

Solution

Solutionの全体像を示す図:



Validation戦略

当初から、多くの参加者と同様に、question idによるgroup splitのCVとpublic LBの間に良い相関を見出せませんでした。そのため、hidden dataにはtrainに存在しないunseen misconceptionが多数含まれていることが明らかになり、train dataが持たないunseen misconceptionの一部を含むようにvalidation splitを作るべきだと考えました。 question idによるgroup splitでも一定の割合のunseen misconceptionは含まれていましたが、通常はfold内の全misconceptionの最大20%程度にとどまっていました。そのため、多くのチームがseen misconceptionへの過学習に陥りました。unseenのものがretrievalの面で大きく抑え込まれていても、全体のCVスコアは良く見えてしまうのです。 例えば、foldが**~80% seen**のデータと**20% unseen**のデータで構成されている場合、seen misconceptionへの深刻な過学習でMRRが65%、unseenのretrievalが不十分でMRRが30%だとすると、MRRの加重平均は 0.65*0.8 + 0.3*0.2 = 0.58 となり、LBが示す値とは大きくかけ離れたものになります。 最終的に、hiddenの分布はseen/unseen misconceptionの観点でかなりバランスが取れているという仮定に基づき、良いCVの鍵はseenとunseenのmisconceptionに等しい重みを与えることだとして、validation戦略を立てました。 実験の

スピードを高く保ち、VMへの出費を抑えるため、実験間では1つのfoldのみをvalidation setとして使い続けました。異なるfold間で結果が非常に似ていたため、この戦略で十分であることがわかりました。

合成データ生成

提供されたtrain dataのみでretriever modelを学習する試みは悪い結果となったため、unseen misconceptionのための合成データ生成は避けられないものになりました。 Qwen 32/72 instructを用いたLORA pretrainingとfew-shot生成を実験する中で、LORA pretrainingはmodelの推論能力を低下させるようだとわかりました（catastrophic forgetting？）。最良のアプローチはfew-shot生成でした。後に、素晴らしいEEDI paperに出会い、このタスクにはfew-shot生成がより効果的だという同様の結論に至っていることを知りました。

few-shot生成の主な欠点は、生成されたサンプルの言い回しがfew-shotの例と非常に似てしまうことでした。prompt engineeringをどう試みても、distractorとmisconceptionの正しい対応関係を失わずにこれを変えることはできませんでした。

生成の詳細:

- Qwen 72B instruct（32Bはトリッキーなmisconceptに苦戦しました）
- 最も類似したtrain misconceptに対する、ランダムな3つのfew-shot例
- train dataの構造に合わせた数学テスト全体の生成。ただし、misconceptに関連するdistractorは1つだけ
- misconceptごとに3つの数学テストを、3つの異なるseedで繰り返し生成。つまり、各misconceptにつき合計9つのテスト
- 全サンプルの生成に2xA40で3時間かかりました

Retrievers

数学問題のLaTex形式には冗長な記号が多く含まれており、収束を遅くしていました。そこで、textを手作業でフィルタリングする代わりに、LaTexをplain textに変換して数式を簡略化するopen-sourceのライブラリを調べました。大半のライブラリは冗長な記号の大部分を無視してしまいましたが、`pylatexenc`ライブラリを見つけました。これは変換をかなりうまく処理し、textや数式から重要な要素をほとんど失いませんでした（training dataには、LaTex形式にまったく従っていない冗長な記号を含むケースがいくつかありました）。ただし、重要な情報が失われないよう、LaTexのフィルタリングを行っていないデータを使った2つのmodelも最終ensembleに含めました。 興味深いのは、生成データを含めて学習した後、（合成データによるoversamplingのおかげで）modelはunseen misconceptionに対してseenのものよりはるかに高いMRRを達成したにもかかわらず、recallはseenとunseenのmisconceptで同じままだったことです。根本的な原因は、合成データによってmodelは単純なmisconceptionにはよく収束できた一方で、複雑なmisconceptについては数学テストの生成がうまくいかなかったことにあると考えています。その結果、modelはunseen misconceptの大半をうまくretrieveできました（高いMRRにつながりました）が、他の複雑なmisconceptについては関連性を見出せず、trainとvalidationのsplit間で数学テストが似ていた、少数しか出現しないseen misconceptと同じrecallになりました。

Retrieverの詳細:

- Models: gte-Qwen2-7B-instruct, Salesforce-SFR-Embedding-2_R, bge-multilingual-gemma2, Qwen2.5-14B-Instruct
- Loraパラメータ:
  - rank = 64
  - alpha = 128
- Batch size = 32

- サンプルあたりのnegative数 = 5（最も類似した0位から30位までのnegativeから）
- 2回のneg miningイテレーション:
  - 最初のneg miningと、それに続く1600 training steps
  - pretrained modelによる2回目のneg miningイテレーションと、それに続く400 training steps
- BNB 4 bit量子化
- 1つのmodelの学習に1xRTX 3090で9時間かかりました

Rerankers

prompt tokenを学習から除外するために、`DataCollatorForCompletionOnlyLM`を使うことが決定的に重要でした。 もう1つの重要な点は、testのmisconceptionをすべて学習から除外することでした。trainingのmisconceptionのみで学習することで、modelがunseen misconceptionに対して効果的に汎化できることを確認しました。testのmisconceptを除外したことは、publicで0.6に到達した後で、おそらく最大のLB改善（~0.61 - 0.63）をもたらしました。

Distillation

締切間近に、興味深いトリックを発見しました。まず、1つのseedでmodelを学習します。次に、別のseedでデータセットを作成します（これにより通常、negative sampleの集合と正解misconceptionの位置が変わります）。学習済みのmodelを使って、この新しいデータセットでlogitsを推論します。最後に、KL divergenceによるdistillationと単純なcross-entropy lossを組み合わせてmodelを再学習します。このアプローチにより、publicで~0.01の改善が得られました。 このトリックの背後にある私の直感は、一部のサンプルでは技術的に複数のmisconceptionが正解になりうる、というものです。modelを単一のmisconceptionに過学習させると、汎化能力が低下します。distillationを通じてsoft labelを取り入れることで、過学習を緩和し、unseen misconceptionに対するmodelの汎化を改善できます。

First stage reranker

最も類似した100件のmisconceptionから49件のnegative misconceptionをランダムにサンプリングし、正解のmisconceptionをランダムな位置に配置しました。そして、どのmisconceptionが最も関連しているかを分類するようにmodelを学習しました。サンプリングによって生じるランダムな順序は、retrieverが正解のmisconceptionを最初の数個の選択肢に置くことが多いという、初期の順序への過学習を避けるのに役立ちました。

Misconceptionの言い換え

misconceptionの書き方が大きく異なることに気づきました。例えば、欠けている知識を記述しているものもあれば、生徒が信じていることを記述しているものもあります。これに対処するため、misconceptionを、欠けている知識とそのmisconceptionに沿った生徒の信念の両方を含む一般的な形式に言い換えました。言い換えたmisconceptionが意味的に元のものを繰り返しているだけの場合でも、この言い換えはmodelの推論時に効果的な正則化として機能しました。 言い換えたmisconceptionでの推論logitsを、元のmisconceptionでのlogitsと組み合わせて使用し、publicで~0.01の改善が得られました。 misconceptionの言い換えにはQwen 32B-Instructを使用しました。

Second stage reranker

first-stage rerankerから得たlogitsを分析したところ、Qwenはretrieverが並べたmisconceptionの初期順序を保持する傾向があることに気づきました。この観察を裏付けるものをこの優れた論文で見つけ、それに触発されて、MRRをより重視してRetrieverを再学習し、first stage rerankerの結果を洗練するsecond-stage rerankerを開発しました。 second stageでは、first stage rerankerの上位にrankされたmisconceptionから小さなwindowを選び、そ

の順序についてあらゆる順列を適用しました。そして、これらの順列にわたるlogitsを平均し、そのwindow内のmisconceptを再度rerankしました。驚いたことに、最適なwindowサイズは2でした。つまり、first stage rerankerの上位2件のmisconceptionを取り出して推論し、その順序を反転させて再度推論し、最後に平均したlogitsを最終的なrerankingに使用しました。 **Rerankerの詳細:**

- Qwen 32B instructを使用
- Loraパラメータ:
  - rank = 64
  - alpha = 128
- Batch size = 2 * 6 grad accum
- AWQ 4 bit量子化
- 1つのmodelの学習に1xA40で6時間かかりました

うまくいかなかったこと

- **非常に多くのこと**
- 利用可能なすべてのlora adapterでfp16のRerankerを学習し、その後に量子化すること
- Qwen 72BをRerankerとして学習すること。32Bと同じ結果でした
- Rerankerの学習に合成データを使うこと

Hardware

ウクライナのインフラは絶え間ない攻撃を受けており、その結果として毎日停電が起きているため、VMを借りることが避けられませんでした。 主にA40を使い、一部の実験ではA100を借りました。 合計で約$300かかりました。